Add Seed Data to Unique ID Config

This API is available on these plans:

✕ Limited ✕ Standard ✓ Advanced

Important

Your access to this API is as per your agreement with Syndigo. For more information on access levels, see Access Levels for RESTFul API. In addition, all the new notification configs must adhere to the current format for the smooth functioning of email notification.

You can use the RESTful API **{TenantURL or ID}/api/configurationappservice/createuniqueidconfig** to add seed data to the **unique ID config**. The application uses this seed data to generate unique IDs.

Note:

Unique ID config is a one-time setup done in the tenant based on your business requirements. You can contact Syndigo Support Team for further support.

The application provides you with an **API** (getuniqueid) and a **keyword** (GenerateUniqueID) to generate the unique IDs in a sequential order of numbering.

Scenario

Consider that you wish to create seed data of a combination of an **entity type** `"sku"`, a **context** `"Australia"`, an additional context of `"Webtaxonomy"`, and a **relationship type** as `"hasimages"` in the **unique ID config**.

To create seed data using **createuniqueidconfig** API, configure the header and use the sample JSON request mentioned below.

Header

The header of the request contains parameters to authenticate and authorize the request. The request is authenticated only if the API authentication is enabled in the tenant configuration. For more information, see Authentication Services.

To authenticate the request, you must:

- Use https and append the request with the current time-stamp in ISO_8601 format (&timeStamp="current timestamp in ISO_8601 format")
- Generate authToken by hashing the request using the clientAuthKey with HmacSHA256 algorithm.
- Pass the clientId, userId, and authToken in the header.

You can also pass additional parameters such as ownershipData and userRoles in the request header to authorize the request. For more information, see [Authorization Model] and [Understanding the Request Headers].

Request

The following is a sample request to create the seed data:

```
{
  "configObject": {
    "id": "sku_australia_hasImages",
    "type": "uniqueIdConfig",
    "properties": {
      "entitytype": "sku",
      "context": "Australia",
      "additionalcontext": "Webtaxonomy",
      "relationshiptype": "hasimages",
      "prefix": "B_",
      "suffix": "",
      "paddirection": "left",
      "padcharacter": "1",
      "totalwidth": 13,
      "startupseed": 0,
      "increment": 1
    }
  }
```

```
}
```

Parameters

The following table contains the parameters and their associated datatypes and descriptions, which are present in the sample JSON request:

Parameters	Type	Mandatory	Description
"entitytype"	String	Yes	The entity type for which you are generating the seed data. For example, “sku”.
"context"	String	No	The context of the seed data. For example, “Australia”.
"additionalcontext"	String	No	The additional context such as classification or taxonomy of the seed data. For example, “Webtaxonomy”.
"relationshiptype"	String	No	The relationship type of the seed data. For example, “hasimages”.
"prefix"	String	No	The prefix of the seed data. For example, “B_”.
"suffix"	String	No	The suffix of the seed data. For example, “”.
"paddirection"	String	No	The padding direction of the seed data. For example, “left”. The supported values are “left” and “right”.
"padcharacter"	String or Integer	No	The padding character of the seed data. For example, “1”. The maximum limit of this parameter is one character.
"totalwidth"	Integer	No	The total width of the seed data. This defines the character

Parameters	Type	Mandatory	Description
			limit for unique ID getting generated. For example, “13”. The range is from “-2147483648” to “+2147483647”.
"startupseed"	Integer	No	The point from when the startupseed is defined for the seed data. For example, “0”. The range is from “-9223372036854775808” to “+9223372036854775807”.
"increment"	Integer	No	The incremental value for the seed data. For example, “1”. The range is from “-9223372036854775808” to “+9223372036854775807”.

Notes:

- All the above-mentioned values for the parameters are user-defined.
- The id sku_australia_hasImages under the configObject is a dummy ID and does not serve any validation for the request.
- The values of the parameters are **case insensitive**. This is to avoid duplication and to maintain the quality of the config data. For example, consider that an entry for the **entity type** “sku” is already present in the **unique ID config**. Now, a request with the **entity type** “SKU” will fail as this request will be considered as a **duplicate** for “sku” **entity type**.
- The **padcharacter** parameter cannot have a value with more than one character. For example, if you give the value “23” which has **two** characters instead of **one**, the API request will fail.

Response

The following is the response for the successful completion of the **createuniqueidconfig** request:

```
{
  "request": {
    "returnRequest": false,
    "requestId": "ff1a7aac-426a-468b-a08d-a5414a4d5a14",
    "taskId": "ff1a7aac-426a-468b-a08d-a5414a4d5a14"
  },
  "response": {
    "status": "success",
    "statusDetail": {
      "messages": [
        {
          "message": "Config data created successfully for entityType : sku , context
: australia, additionalcontext : Webtaxonomy and relationship : hasimages",
          "messageCode": "I0161",
          "messageType": "success"
        }
      ]
    },
    "totalRecords": 0
  }
}
```


If a user tries to add config data that is existing in the system:


```
{
  "response": {
    "status": "success",
    "statusDetail": {
      "messages": [
        {
          "message": "Config data already exists for entityType : sku , context :
australia and relationship : hasImages ",
```

```
          "messageCode": "W0461",
          "messageType": "success"
        }
      ]
    },
    "totalRecords": 0
  }
}
```

Verification

You can use the [getuniqueidconfig] API to verify the successful creation of the config data.

Generate Unique ID Scenarios

A unique ID gets generated as per the values specified in the “prefix”, “suffix”, “paddirection”, “padcharacter”, “totalwidth”, “startupseed”, and “increment” parameters.

The following are the success scenarios:

Scenario 1: Consider the following sample JSON request:

```
{
  "configObject": {
    "id": "sku_australia_hasImages",
    "type": "uniqueIdConfig",
    "properties": {
      "entitytype": "sku",
      "context": "Australia",
      "additionalcontext": "productclassification",
      "relationshiptype": "hasimages",
      "prefix": "B_",
```

```
      "suffix": "",
      "paddirection": "left",
      "padcharacter": "#",
      "totalwidth": 13,
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

Result: The unique ID would be `“##########B_1"`. The generated unique ID is `“13”` characters in length as the value specified for the `“totalwidth”` parameter is `“13”`.

Here, the unique ID starts with `“B_”` since it is the `“prefix”` value defined and followed by `“1”` as the `“startupseed”` is `“0”`, which is incremented by `“1”`- value of the `“increment”` parameter. A null or zero value is considered for `“suffix”` since no value is specified in the sample JSON. The series of `“#”` in the unique ID is the `“padcharacter”` value that starts from the `“left”`, as defined in the `“paddirection”` parameter. This `“padcharacter”` value helps to fill or satisfy the remaining byte-length of the unique ID.

Scenario 2: Consider the following sample JSON request with different input values:

```
{
  "configObject": {
    "id": "sku_australia_hasImages",
    "type": "uniqueIdConfig",
    "properties": {
      "entitytype": "sku",
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "B_",
      "suffix": "S",
```

```
      "paddirection": "right",
      "padcharacter": "*",
      "totalwidth": 13,
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

Result: Upon execution, the generated unique ID will be `“B_1S*********”`.
Here, `“B_”` is the `“prefix”` value, `“1”` is the `“increment”` value, `“S”` is the `“suffix”` value, and the series of `“*”` is the `“padcharacter”` added to the right. The length of this unique ID is `“13”` characters, as specified in the `“totalwidth”` parameter in the sample request JSON.

Scenario 3: Consider the following sample JSON request with different input values:

```
{
  "configObject": {
    "id": "sku_australia_hasImages",
    "type": "uniqueIdConfig",
    "properties": {
      "entitytype": "sku",
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "B_",
      "suffix": "S",
      "paddirection": "right",
      "padcharacter": "*",
      "totalwidth": 10,
      "startupseed": 101,
      "increment": 1
    }
```

```
  }
}
```

Result: Upon execution, the generated unique ID will be `“B_102S****”`. Here, `“B_”` is the `“prefix”` value, `"102"` is the result of incrementing the `"startupseed"` value, which is `"101`, by 1 (the `"increment"` parameter), `“S”` is the `“suffix”` value, and the series of `“*”` is the `“padcharacter”` added to the right. The length of this unique ID is `“10”` characters, as specified in the `“totalwidth”` parameter.

Error Scenarios

The following are some example requests that can result in a response error.

Scenario 1: When “type” is “uniqueIdtype” or missing under “configObject”:

Request

```
{
  "configObject": {
    "id": "sku_australia_hasimages",
    "type": "uniqueIdtype",
    "properties": {
      "entitytype": "sku",
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "SKU",
      "suffix": "",
      "paddirection": "right",
      "padcharacter": "&",
      "totalwidth": 20,
      "startupseed": 1,
      "increment": 1
    }
  }
```

```
}
```

The following is the response error when “type” is “uniqueIdtype” instead of “uniqueIdconfig”:

```
{
  "response": {
    "status": "error",
    "statusDetail": {
      "messages": [
        {
          "message": "Invalid config type",
          "messageCode": "E0137",
          "messageType": "error"
        }
      ]
    },
    "totalRecords": 0
  }
}
```

Scenario 2: Mandatory parameter `“entitytype”` missing from the request.

Request

```
{
  "configObject": {
    "id": "sku_australia_hasImages",
    "type": "uniqueIdConfig",
    "properties": {
      "context": "Australia",
      "relationshiptype": "hasimages",
```

```
      "prefix": "B_",
      "suffix": "",
      "paddirection": "left",
      "padcharacter": "10",
      "totalwidth": 13,
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

The following is the response error for the missing mandatory parameter in the request:

```
{
  "response": {
    "status": "error",
    "statusDetail": {
      "messages": [
        {
          "message": "Invalid request, entitytype is null or empty",
          "messageCode": "E0133",
          "messageType": "error"
        }
      ]
    },
    "totalRecords": 0
  }
}
```

Scenario 3: An incorrect paddirection value, for example “up” or “down”, is specified for a specific entityType:

Request

```
{
  "configObject": {
    "id": "sku_australia_hasImages",
    "type": "uniqueIdConfig",
    "properties": {
      "entitytype": "SKU",
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "B_",
      "suffix": "",
      "paddirection": "up",
      "padcharacter": "1",
      "totalwidth": 13,
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

The following is the response error for the unsupported paddirection value in the request:

```
{
  "response": {
    "status": "error",
    "totalRecords": 0,
    "statusDetail": {
      "messages": [
```

```
        {
          "message": "Invalid request, paddirection should be left or right",
          "messageCode": "E0134",
          "messageType": "error"
        }
      ]
    }
  }
}
```

Scenario 4: Incorrect parameter "padcharacter" value “10” given in the request:

Request

```
{
  "configObject": {
    "id": "sku_australia_hasImages",
    "type": "uniqueIdConfig",
    "properties": {
      "entitytype": "SKU",
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "B_",
      "suffix": "",
      "paddirection": "left",
      "padcharacter": "10",
      "totalwidth": 13,
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

The following is the response error for the incorrect “padcharacter” value in the request:

```
{
  "response": {
    "status": "error",
    "statusDetail": {
      "messages": [
        {
          "message": "Invalid request, padcharacter length should be one",
          "messageCode": "E0135",
          "messageType": "error"
        }
      ]
    },
    "totalRecords": 0
  }
}
```

Scenario 5: The following error response is obtained when there is a database connection or PostgreSQL connection failure:

```
{
  "response": {
    "status": "error",
    "totalRecords": 0,
    "statusDetail": {
      "messages": [
        {
          "message": "Internal processing failure, please contact support ",
          "messageCode": "E0136",
          "messageType": "error"
```

```
        }
      ]
    }
  }
}
```

Update Seed Data to Unique ID Config

This API is available on these plans:

✕ Limited ✕ Standard ✓ Advanced

Important

Your access to this API is as per your agreement with Syndigo. For more information on access levels, see Access Levels for RESTFul API. In addition, all the new notification configs must adhere to the current format for the smooth functioning of email notification.

You can use the RESTful API **{TenantURL or ID}/api/configurationappservice/updateuniqueidconfig** to update seed data in the **unique ID config**. The application uses this seed data to update existing unique IDs.

Note:

Unique ID config is a one-time setup done in the tenant based on your business requirements. You can contact Syndigo Support Team for further support.

The application provides you with an **API** (getuniqueid) and a **keyword** (GenerateUniqueID) to generate the unique IDs in a sequential order of numbering. You can then use the getuniqueid API to update the existing unique ID.

Scenario

Consider that you wish to update seed data of a combination of an **entity type** “sku”, a **context** “Australia”, an additional context of “Webtaxonomy”, and a **relationship type** as “hasimages” in the **unique ID config**.
To update seed data using **updateuniqueidconfig** API, specify the following in the request:

Parameters	Type	Mandatory	
`"entitytype"`	String	Yes	The entity type for which you are updating the
`"context"`	String	No	The context of the seed data. For example, “A
`"additionalcontext"`	String	No	The additional context such as classification o
`"relationshiptype"`	String	No	The relationship type of the seed data. For ex
`"prefix"`	String	No	The prefix of the seed data. For example, “B_”
`"suffix"`	String	No	The suffix of the seed data. For example, “”.
`"paddirection"`	String	No	The padding direction of the seed data. For ex
`"padcharacter"`	String or Integer	No	The padding character of the seed data. For e
`"totalwidth"`	Integer	No	The total width of the seed data. This defines t example, “13”. The range is from “-214748364
`"startupseed"`	Integer	No	The point from when the `startupseed` is defined 9223372036854775808” to “+9223372036854
`"increment"`	Integer	No	The incremental value for the seed data. For e “+9223372036854775807”.

Notes:

• All the above-mentioned values for the parameters are user-defined.
• Any change in the input of “entitytype”, “context”, or ”relationshiptype” parameters result in creating a new record for that config data.
• For any existing config data, any change in the input of “prefix”, “suffix”, “paddirection”, “padcharacter”, “totalwidth”, “startupseed”, or “increment” parameters results in updating that config data.

Header

The header of the request contains parameters to authenticate and authorize the request. The request is authenticated only if the API authentication is enabled in the tenant configuration. For more information, see Authentication Services.

To authenticate the request, you must:

- Use https and append the request with the current time-stamp in ISO_8601 format (&timeStamp="current timestamp in ISO_8601 format")
- Generate authToken by hashing the request using the clientAuthKey with HmacSHA256 algorithm.
- Pass the clientId, userId, and authToken in the header.

You can also pass additional parameters such as ownershipData and userRoles in the request header to authorize the request. For more information, see Authorization Model and Understanding the Request Headers.

Success scenarios

Scenario 1: The following is a sample request to update seed data:
Request 1

```
{
 "configObject": {
   "id": "sku_australia_hasImages",
   "type": "uniqueIdConfig",
   "properties": {
     "entitytype": "sku",
     "context": "Australia",
     "additionalcontext": "Webtaxonomy",
     "relationshiptype": "hasimages",
     "prefix": "B_",
     "suffix": "",
     "paddirection": "left",
     "padcharacter": "1",
     "totalwidth": 13,
```

```
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

Notes:

• The id sku_australia_hasImages under the configObject is a dummy ID and does not serve any validation for the request.
• The values of the parameters are case insensitive. This is to avoid duplication and to maintain the quality of the config data. For example, consider that an entry for the entity type “sku” is already present in the unique ID config. Now, a request with the entity type “SKU” will fail as this request will be considered as a duplicate for “sku” entity type.
• The padcharacter parameter cannot have a value with more than one character. For example, if you provide the value “23”, which has two characters instead of one, the API request will fail.

Response 1

The following is the response for the successful completion of the **updateuniqueidconfig** request:

```
{
  "request": {
    "returnRequest": false,
    "requestId": "ff1a7aac-426a-468b-a08d-a5414a4d5a14",
    "taskId": "ff1a7aac-426a-468b-a08d-a5414a4d5a14"
  },
  "response": {
    "status": "success",
    "statusDetail": {
      "messages": [
        {
```

```
          "message": "Config data updated successfully for entityType : sku , context
: australia, additionalcontext : Webtaxonomy and relationship : hasimages",
          "messageCode": "I0163",
          "messageType": "success"
        }
      ]
    },
    "totalRecords": 0
  }
}
```


Notes:

The above response is obtained when:
• A user tries to update config data that does or does not exist in the application.
• A user tries to update non-existing config data, the API functions like CREATE endpoint and creates a new record for that config data.

Scenario 2: A user executes this API with no change in the input values of the parameters.
Result: The following response is obtained.

Response 2:


```
{
  "response": {
    "status": "success",
    "statusDetail": {
      "messages": [
        {
          "message": " Config data has no change, hence not updating the data for
entityType : sku, context : australia and relationship : hasImages ",
          "messageCode": "W0462",
          "messageType": "success"
        }
```

```
      ]
    },
    "totalRecords": 0
  }
}
```

Verification

You can use the [getuniqueidconfig](#) API to verify the successful creation of the config data.

Error Scenarios

The following are some of the sample error scenarios.

Scenario 1: When “type” is “uniqueIdtype” under “configObject”.

Request - Error Scenario 1

```
{
  "configObject": {
    "id": "sku_australia_hasimages",
    "type": "uniqueIdtype",
    "properties": {
      "entitytype": "sku",
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "SKU",
      "suffix": "",
      "paddirection": "right",
      "padcharacter": "&",
      "totalwidth": 20,
      "startupseed": 1,
      "increment": 1
```

```
    }
  }
}
```

The following is the response error when “type” is “uniqueIdtype” instead of “uniqueIdconfig”:

Response - Error Scenario 1

```
{
  "response": {
      "status": "error",
      "statusDetail": {
        "messages": [
          {
            "message": "Invalid config type",
            "messageCode": "E0137",
            "messageType": "error"
          }
        ]
      },
      "totalRecords": 0
  }
}
```

Scenario 2: Sample request when “uniqueIdconfig” is missing for the “type” parameter under “configObject”:

Request - Error Scenario 2

```
{
  "configObject": {
```

```
      "id": "sku_australia_hasimages",
      "properties": {
        "entitytype": "sku",
        "context": "germany",
        "relationshiptype": "hasImages",
        "prefix": "SKU",
        "suffix": "",
        "paddirection": "right",
        "padcharacter": "&",
        "totalwidth": "20",
        "startupseed": "1",
        "increment": "1"
    }
  }
}
```

Response - Error Scenario 2

The following is the response error for the missing “uniqueIdconfig” in the request:

```
{
  "response": {
    "status": "error",
    "statusDetail": {
      "messages": [
        {
          "message": "Invalid config type",
          "messageCode": "E0137",
          "messageType": "error"
        }
      ]
    },
```

```
    "totalRecords": 0
  }
}
```

Scenario 3: Mandatory parameter “entitytype” missing from the request.

Request - Error Scenario 3

```
{
    "configObject": {
        "id":"sku_australia_hasImages",
        "type": "uniqueIdConfig",
      "properties": {
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "B_",
      "suffix": "",
      "paddirection": "left",
      "padcharacter": "1",
      "totalwidth": 13,
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

Response - Error Scenario 3

The following is the response error for the missing mandatory parameter in the request:

```
{
  "response": {
```

```
    "status": "error",
    "statusDetail": {
      "messages": [
        {
          "message": "Invalid request, entitytype is null or empty",
          "messageCode": "E0133",
          "messageType": "error"
        }
      ]
    },
    "totalRecords": 0
  }
}
```


Scenario 4: An incorrect paddirection value, for example “up” or “down”, is specified for a specific entityType.

Request - Error Scenario 4


```
{
    "configObject": {
        "id":"sku_australia_hasImages",
        "type": "uniqueIdConfig",
      "properties": {
      "entitytype": "SKU",
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "B_",
      "suffix": "",
      "paddirection": "up",
      "padcharacter": "1",
      "totalwidth": 13,
```

```
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

The following is the response error for the unsupported paddirection value in the request:

Response - Error Scenario 4

```
{
"response": {
        "status": "error",
        "totalRecords": 0,
        "statusDetail": {
            "messages": [
                {
                    "message": "Invalid request, paddirection should be left or
right",
                    "messageCode": "E0134",
                    "messageType": "error"
                }
            ]
        }
    }
}
```

Scenario 5: Incorrect parameter "padcharacter" value “10” given in the request:

Request - Error Scenario 5

```
{
```

```
    "configObject": {
        "id":"sku_australia_hasImages",
        "type": "uniqueIdConfig",
    "properties": {
      "entitytype": "SKU",
      "context": "Australia",
      "relationshiptype": "hasimages",
      "prefix": "B_",
      "suffix": "",
      "paddirection": "left",
      "padcharacter": "10",
      "totalwidth": 13,
      "startupseed": 0,
      "increment": 1
    }
  }
}
```

Response - Error Scenario 5

The following is the response error for the incorrect “padcharacter” value in the request:

```
{
  "response": {
    "status": "error",
    "statusDetail": {
      "messages": [
        {
          "message": "Invalid request, padcharacter length should be one",
          "messageCode": "E0135",
          "messageType": "error"
```

```
        }
      ]
    },
    "totalRecords": 0
  }
}
```

Scenario 6: The following error response is obtained when there is a database connection or PostgreSQL connection failure.

Response - Error Scenario 6

```
{
  "response": {
    "status": "error",
    "totalRecords": 0,
    "statusDetail": {
      "messages": [
        {
          "message": "Internal processing failure, please contact support ",
          "messageCode": "E0136",
          "messageType": "error"
        }
      ]
    }
  }
}
```

Get Seed Data from Unique ID Config

This API is available on these plans:

✕ Limited ✕ Standard ✓ Advanced

Important

Your access to this API is as per your agreement with Syndigo. For more information on access levels, see Access Levels for RESTFul API. In addition, all the new notification configs must adhere to the current format for the smooth functioning of email notification.

You can use the RESTful API **{TenantURL or ID}/api/configurationappservice/getuniqueidconfig** to get an entity type's seed data from the **unique ID config**. For more information, see Add Seed Data API to Unique ID Config.

Scenario

Consider you wish to get the **sku** entity type's seed data from the **unique ID config**.

In the API request, specify the following:

- `"entitytype"` – Specify the entity type value `"sku"`. This is a **mandatory** parameter.

Note:

If you want to get the seed data for all entity types, then, you can give the value `"_ALL"` for `"entitytype"` parameter in the API request.

Header

The header of the request contains parameters to authenticate and authorize the request. The request is authenticated only if the API authentication is enabled in the tenant configuration. For more information, see Authentication Services.

To authenticate the request, you must:

- Use https and append the request with the current time-stamp in ISO_8601 format (&timeStamp="current timestamp in ISO_8601 format")
- Generate authToken by hashing the request using the clientAuthKey with HmacSHA256 algorithm.
- Pass the clientId, userId, and authToken in the header.

You can also pass additional parameters such as ownershipData and userRoles in the request header to authorize the request. For more information, see Authorization Model and Understanding the Request Headers.

Request

The following is a sample request to get seed data related to the **sku** entity type:

```
{
  "params": {
    "query": {
      "filters": {
        "typesCriterion": ["uniqueIdConfig"],
        "propertiesCriterion": [
          {
            "entityType": {
              "exact": "_ALL"
            }
          }
        ]
      }
    }
  }
}
```

Response

The following is the response for the successful completion of the **getuniqueidconfig** request:


```
{
  "response": {
    "entities": [
      {
        "properties": {
          "entitytype": "_ALL",
          "context": "Germany",
          "additionalcontext": "Webtaxonomy",
          "relationshiptype": "hasImages",
          "prefix": "SGH",
          "suffix": "",
          "paddirection": "left",
          "padcharacter": "0",
          "totalwidth": "1",
          "startupseed": "1",
          "increment": "1",
          "lastgeneratedid": ""
        }
      }
    ],
    "status": "success",
    "totalRecords": 1
  }
}
```


Error Scenarios

The following are some example requests that can result in a response error:

Scenario 1: The following is the response error for the missing unique ID config in the request:

```
{
  "response": {
    "status": "success",
    "totalRecords": 0,
    "statusDetail": {
      "messages": [
        {
          "message": "Invalid config type",
          "messageCode": "E0137",
          "messageType": "error"
        }
      ]
    }
  }
}
```

Scenario 2: Mandatory parameter “entitytype” missing or misspelled in the request.

```
{
  "params": {
    "query": {
      "filters": {
        "typesCriterion": ["uniqueidconfig"],
        "propertiesCriterion": [
          {
            "entityType": {
              "exact": ""
            }
          }
```

```
        ]
      }
    }
  }
}
```

The following is the response error for the missing mandatory parameter in the request:

```
{
  "response": {
    "status": "error",
    "totalRecords": 0,
    "statusDetail": {
      "messages": [
        {
          "message": "Invalid request, entitytype is null or empty",
          "messageCode": "E0133",
          "messageType": "error"
        }
      ]
    }
  }
}
```

Scenario 3: An internal server failure occurs such as database connection failure:

```
{
  "response": {
    "status": "error",
    "totalRecords": 0,
    "statusDetail": {
```

```
      "messages": [
        {
          "message": "Internal processing failure, please contact support",
          "messageCode": "E0136",
          "messageType": "error"
        }
      ]
    }
  }
}
```

Delete Seed Data from Unique ID Config

This API is available on these plans:

✕ Limited ✕ Standard ✓ Advanced

Important

Your access to this API is as per your agreement with Syndigo. For more information on access levels, see Access Levels for RESTFul API. In addition, all the new notification configs must adhere to the current format for the smooth functioning of email notification.

You can use the RESTful API **{TenantURL or ID}/api/configurationappservice/deleteuniqueidconfig** to delete seed data from the unique ID config. The application uses this seed data to delete an existing unique ID.

The application provides you with an **API** (getuniqueid) and a **keyword** (GenerateUniqueID) to generate the unique IDs in a sequential order of numbering. You can then use this API to delete existing unique ID.

Scenario

Consider that you wish to delete seed data of a combination of an entity type “sku”, a context “Australia”, and a relationship type “hasimages” in the unique ID config.
To delete seed data using deleteuniqueidconfig API, specify the following in the request:

Parameters	Type	Mandatory	
`"entitytype"`	String	Yes	The entity type for which you are deleting the
`"context"`	String	No	The context of the seed data. For example, “A
`"additionalcontext"`	String	No	The additional context such as classification o
`"relationshiptype"`	String	No	The relationship type of the seed data. For ex
`"prefix"`	String	No	The prefix of the seed data. For example, “B_”
`"suffix"`	String	No	The suffix of the seed data. For example, “”.
`"paddirection"`	String	No	The padding direction of the seed data. For ex
`"padcharacter"`	String or Integer	No	The padding character of the seed data. For e
`"totalwidth"`	Integer	No	The total width of the seed data. This defines t example, “13”. The range is from “-214748364
`"startupseed"`	Integer	No	The point from when the `startupseed` is defined 9223372036854775808” to “+9223372036854
`"increment"`	Integer	No	The incremental value for the seed data. For e “+9223372036854775807”.

Note:

All the above-mentioned values for the parameters are user-defined.

Header

The header of the request contains parameters to authenticate and authorize the request. The request is authenticated only if the API authentication is enabled in the tenant configuration. For more information, see Authentication Services.

To authenticate the request, you must:

- Use https and append the request with the current time-stamp in ISO_8601 format (&timeStamp="current timestamp in ISO_8601 format")
- Generate authToken by hashing the request using the clientAuthKey with HmacSHA256 algorithm.
- Pass the clientId, userId, and authToken in the header.

You can also pass additional parameters such as ownershipData and userRoles in the request header to authorize the request. For more information, see Authorization Model and Understanding the Request Headers.

Scenario 1: Delete seed data

Request

The following is a sample request to delete seed data:

```
{
  "configObject": {
      "id":"sku_australia_hasImages"
      "type": "uniqueIdConfig",
  "properties": {
    "entitytype": "sku",
    "context": "Australia",
    "relationshiptype": "hasimages",
    "prefix": "B_",
    "suffix": "",
    "paddirection": "left",
    "padcharacter": "1",
    "totalwidth": 13,
    "startupseed": 0,
    "increment": 1
  }
}
```

```
}
```

Notes:

• The id sku_australia_hasImages under the configObject is a dummy ID and does not serve any validation for the request.
• The values of the parameters are case insensitive. This is to avoid duplication and to maintain the quality of the config data.

Response

The following is the response for the successful execution of the **deleteuniqueidconfig** request:


```
{
  "request": {
    "returnRequest": false,
    "requestId": "ff1a7aac-426a-468b-a08d-a5414a4d5a14",
    "taskId": "ff1a7aac-426a-468b-a08d-a5414a4d5a14"
  },
  "response": {
    "status": "success",
    "statusDetail": {
      "messages": [
        {
          "message": " Config data deleted successfully for entityType : sku ,
context : australia and relationship : hasimages",
          "messageCode": "I0164",
          "messageType": "success"
        }
      ]
    },
    "totalRecords": 0
  }
```

```
}
```

Verification

You can use the **getuniqueidconfig** API to verify the successful creation of the config data.

Sample Error Scenarios

The following are some of the sample error scenarios
Scenario 1: When a user tries to delete a non-existing seed data:
Response:


```
{
    "request": {
        "returnRequest": false,
        "requestId": "badb9428-2db1-4fae-abdb-16bc8fb701e3",
        "taskId": "badb9428-2db1-4fae-abdb-16bc8fb701e3"
    },
    "response": {
        "status": "success",
        "statusDetail": {
            "messages": [
                {
                    "message": "Config data for entityType : sku , context :
australia and relationship : hasimages is not present, hence delete action result is
nochange",
                    "messageCode": "W0463",
                    "messageType": "success"
                }
            ]
        },
        "totalRecords": 0
```

```
    }
}
```

Scenario 2: The following error response is obtained when there is a database connection or PostgreSQL connection failure:

Response:

```
{
    "response": {
        "status": "error",
        "totalRecords": 0,
        "statusDetail": {
            "messages": [
                {
                    "message": "Internal processing failure, please contact support
",
                    "messageCode": "E0136",
                    "messageType": "error"
                }
            ]
        }
    }
}
```